

Robust Control of Ephemeral Weight Dynamics

Jaden Lorenc

Abstract—Fast weights are a small subset of a neural network’s parameters endowed with high learning rates and rapid decay and can serve as a form of short-term memory, encoding recent inputs directly into the parameters of the network rather than into hidden activations. In our prior work on Ephemeral Weights, this mechanism was shown to support character-level recall tasks without recurrent connections, but its training dynamics remained delicate: too little plasticity and the network fails to encode anything; too much and the gradient norms diverge and the model collapses. This paper treats the plasticity hyperparameter α as a control input and the ephemeral weight norm $\|w_{\text{eph}}\|$ as the controlled state, formulating the stability of fast-weight training as a robust control problem. We linearize the fast-weight update equations around an operating point to obtain a scalar plant in the partitioned generalized form, identify its coefficients via gray-box system identification on training traces, and synthesize state-feedback controllers using LQR, H_∞ , and a model-free adaptive heuristic. We compare the three on the palindrome-reversal character prediction task and what worked, what didn’t, and what the failure modes tell us about whether the Riccati-style framework is the right tool for keeping fast-weight networks inside the narrow corridor between forgetting and explosion.

I. INTRODUCTION

While it is typical to consider the loss function and backpropagation as distinct from the forward-pass components of an artificial neural network, our prior work demonstrated that error computation and backpropagation can be repurposed as a form of short-term memory. By designating a small fraction of parameters as *ephemeral weights* — endowed with elevated learning rates and rapid decay — a network can encode recent gradient updates directly into its parameters, replacing the need for recurrent connections or context windows at inference time. This creates a self-organizing memory system where parameter dynamics cluster into distinct functional regimes: stable slow weights for long-term knowledge, and volatile ephemeral weights for working memory.

The challenge is that this volatility is not free. Fast weights live near the boundary of gradient instability by design, the same property that makes them useful as memory (rapid response to incoming gradient signals) is also what threatens to destabilize training entirely. Our prior work established that there is a narrow *corridor* of

effective parameter values: high enough plasticity to encode memory, low enough to avoid the cascading failures that result when high-magnitude weights amplify their own subsequent gradient updates. Traditional stabilization techniques like gradient clipping, batch normalization, layer normalization all actively harm performance by smoothing away the bimodal gradient distributions that the memory mechanism depends on.

In this paper we ask: can the management of fast-weight plasticity itself be approached as a robust control problem, where the plasticity hyperparameter α is treated as a control input chosen at each training step to keep the ephemeral weight norm inside the safe corridor? We linearize the analytical fast-weight update around an operating point to obtain a scalar plant in the partitioned generalized-plant form

$$\delta x(t+1) = A\delta x(t) + B\delta u(t) + E\delta d(t),$$

identify A from sysid traces while taking B and E from analytical formulas, and synthesize feedback controllers using LQR, H_∞ , and a model-free adaptive heuristic that watches the loss directly. We then compare the three on a character-level palindrome reversal task and document, with intentional honesty about negative results, where each approach helps and where it falls short.

Contributions of this paper: - A linearization of the ephemeral-weights fast-weight update suitable for state-space control synthesis. - A gray-box system identification procedure that combines analytical structure derived from the update equation with least-squares fitting on training traces. - Implementations of LQR, H_∞ , and a model-free adaptive controller wrapped around a live training run of an ephemeral-weights RNN. - An empirical comparison of the three controllers on the palindrome-reversal task. - An honest accounting of which assumptions of classical robust control hold for this problem and which fail — and a discussion of whether MPC, μ -synthesis, or simply staying with the adaptive heuristic is the more sensible next move.

II. RELATED WORK

Most of this is lifted with light editing from the Ephemeral Weights paper’s related work section. Two pieces are new and need filling in: (i) prior work on *control of training dynamics* (the closest neighbors to what this paper is doing) and (ii) the dynamic-evaluation lineage

as a precedent for “training-time updates at inference time.”

A. Fast Weights and Parameter-Based Memory

Existing techniques encode memory either in activations (context windows, recurrent connections, external memory modules) or in parameters (pretraining, fine-tuning, hypernetworks, fast weights). Ephemeral Weights belongs to the second category and inherits a long lineage.

Fast weights date back to Hinton (1987) [cite hintonUsingFastWeights1987], who described a dual weight architecture in which each weight is mirrored by one with a higher learning rate that decays toward zero, originally intended to *deblur* old associations rather than to explicitly store short-term information. Schmidhuber (1992) [cite schmidhuberLearningControlFastweight1992] developed the idea further, drawing on biological short-term plasticity to motivate secondary weights that update based on recent activity. Miconi (2018) [cite miconi-DifferentiablePlasticityTraining2018a] implemented differentiable Hebbian plasticity, in which the *degree* of plasticity is itself learned through gradient descent. Ba et al. (2016) [cite baUsingFastWeights2016] generated fast weights from a hidden recurrent connection and decaying previous fast weights, replacing the hidden vector with one calculated through an iterative inner loop using a fast memory weight matrix.

Hypernetworks generalize this further by using one network to generate the weights of another. Fast Weight Programmers [cite irieGoingLinearTransformers2021a] apply this idea to compute weight changes at each step, but introduce efficiency challenges through their outer/inner loop structure. Fast Weight Layers [cite clark-et-al-2022-meta] emulate dynamic evaluation via learned linearized gradient updates, adding an additional neural module on top of the base model.

Closest to our setup, dynamic evaluation [cite mikolovRecurrentNeuralNetwork2010, pmlr-v80-krause18a] performs extra gradient updates on recent data during inference to improve next-token predictions, achieving lower perplexity at the cost of $3\times$ or more inference compute. Ephemeral Weights can be understood as a structurally cleaner version of the same idea: rather than running an outer dynamic-evaluation loop alongside an unmodified network, we designate the parameters whose updates serve as memory directly.

B. Robust Control of Learning Dynamics

[EXPAND — this is the gap I should be filling.] I’m not aware of a body of work that specifically applies H_∞ or μ -synthesis to the control of hyperparameters during neural network training. The closest neighbors are: - Adaptive learning rate schedules (Adam, RMSProp, AdaGrad). These are *implicitly* control mechanisms — they adjust per-parameter learning rates based on gradient statistics — but are not framed as such and have

no formal stability guarantees against worst-case disturbances. - Meta-learning approaches like MAML [cite finn-ModelagnosticMetalearningFast2017], which produce a model with an optimal initialization for fast adaptation but do not perform online control of plasticity. - Reinforcement learning for hyperparameter scheduling (population-based training, etc.) — model-free in spirit, very different in flavor from Riccati synthesis. - **[TODO]** Search for: “Lyapunov stability of gradient descent”, “control-theoretic analysis of SGD”, “adaptive control of learning rate”. There may be work I should be citing from the optimization-as-dynamical-system literature.

C. Robust Control Background

The control side of this paper draws on standard material from MIT 6.241J (Dahleh, Dahleh & Verghese), specifically chapters 15–19 on signal/system norms, feedback interconnection, and uncertainty representation [cite course readings], plus the foundational results of Doyle (1978) [cite Doyle1978LQGMargins], the DGKF H_∞ paper [cite DoyleGloverKhargonekarFrancis1989], and Zames (1981) [cite Zames1981]. For μ -synthesis, which is not covered in the course readings, the canonical references are Doyle (1982) [cite Doyle1982mu] and Packard & Doyle (1993) [cite PackardDoyle1993mu]; an accessible textbook treatment appears in Skogestad & Postlethwaite (2005) chapter 8 [cite SkogestadPostlethwaite2005].

III. BACKGROUND: EPHEMERAL WEIGHTS

Most of this is lifted from the Ephemeral Weights paper. Trim aggressively for length — the *control* paper only needs enough background that the reader understands what we’re controlling.

A. The Ephemeral Weights Mechanism

Each weight w_k in the network is paired with a plasticity parameter α_k determining how quickly it responds to gradient signals. Parameters are divided into two categories: 1. **Slow weights** ($\alpha_k = 1$): standard parameters that learn long-term patterns through ordinary backpropagation. 2. **Ephemeral weights** ($\alpha_k = 10^4$ or 10^5): highly plastic parameters that rapidly encode recent information.

Approximately 10% of the total parameters are randomly designated as ephemeral, excluding the final output layer. The weight update rule is

$$w_k(t+1) = w_k(t) - \alpha_k \nabla_k L(t),$$

where $\nabla_k L(t)$ is computed via Direct Feedback Alignment (DFA) [cite noklandDirectFeedbackAlignment] rather than full backpropagation. DFA provides a stable gradient signal regardless of the volatile state of the high-plasticity parameters; if we were to use backpropagation, the high-plasticity weights would themselves participate

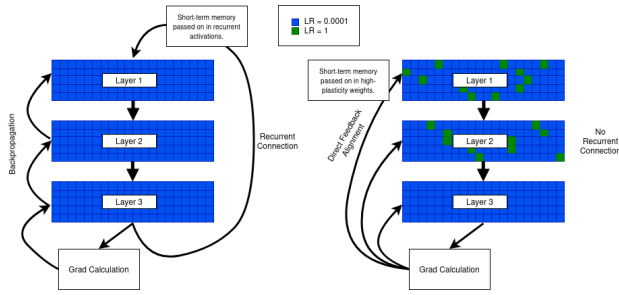


Fig. 1. Block diagram of the ephemeral weights model contrasted with a standard RNN. The recurrent connection h_{t-1} is replaced by short-term storage in the rapidly-updating parameters themselves.

in the backward pass and introduce extreme volatility into the learning dynamics.

B. Forgetting Rate

To prevent gradient explosion from rapid weight changes, the ephemeral weights are subjected to a controlled per-step decay:

$$w_k^{\text{fast}}(t+1) \leftarrow \gamma w_k^{\text{fast}}(t+1),$$

with $\gamma_0 = 0.7$ as the nominal forgetting coefficient. This ensures that information stored in ephemeral weights gradually fades, maintaining stability while allowing the network to adapt to new inputs.

C. Why this needs control in the first place

In our prior work we observed a characteristic failure mode: the gradient norms of high-plasticity weights would gradually grow until they suddenly cascaded into a runaway, with both gradient norms and weight magnitudes accelerating simultaneously, the loss spiking, and accuracy collapsing. In stable training runs, the *ratio* of gradient norms between high-plasticity and low-plasticity weights gradually shrinks; in runs that eventually explode, this ratio grows from the start.

A hyperparameter sweep on the palindrome reversal task showed that the linear interaction of the base learning rate and the high-plasticity learning rate determines both performance and stability. There is a *corridor* of effective parameter values: high enough plasticity to enable effective memory encoding, low enough to avoid blow-up. Hand-tuning α within this corridor is possible but brittle — the corridor narrows as training progresses and the loss landscape changes shape.

The natural question — and the one this paper asks — is whether the traversal of this corridor can be automated by treating α as a control input and the ephemeral weight norm as the state of a linearized plant.

IV. A QUICK ROBUST CONTROL PRIMER

Cap this section at ~ 1.5 pages. State, don't derive. Cite the textbook chapter at the end of each subsection.

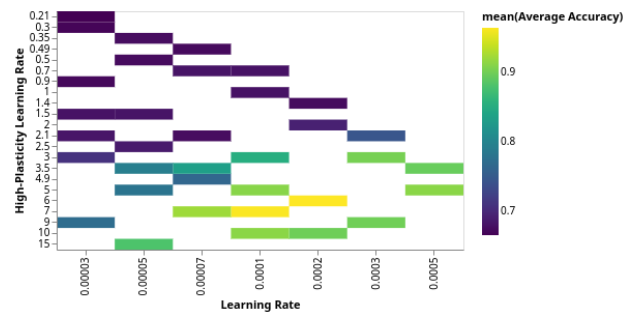


Fig. 2. Hyperparameter sweep showing the corridor between fail-to-memorize (low α) and explosion (high α).

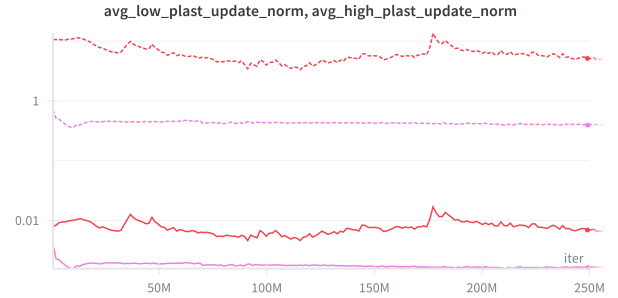


Fig. 3. Gradient norm evolution showing the bimodal regime: high-plasticity weights (red) versus low-plasticity weights (blue).

A. Induced norms and system norms

For a linear operator $T : X \rightarrow Y$ between normed spaces, the *induced norm* is the worst-case amplification of input magnitude into output magnitude:

$$\|T\| := \sup_{x \neq 0} \frac{\|Tx\|_Y}{\|x\|_X}.$$

For a matrix, this is the largest singular value. For a linear time-invariant *system* — a transfer function $G(s)$ that maps input signals to output signals — induced norms become *system norms*. The two natural choices on the input/output signal spaces give the two system norms used throughout robust control: the H_2 norm and the H_∞ norm. (MIT 6.241J chap 15, 16.)

B. The H_2 norm

The H_2 norm is the root-mean-square gain across all frequencies:

$$\|G\|_{H_2}^2 := \frac{1}{2\pi} \int_{-\infty}^{\infty} \text{tr}(G^*(j\omega)G(j\omega)) d\omega.$$

Equivalently, it is the total energy of the system's impulse response, or the steady-state RMS of the output when the input is unit white noise. Minimizing $\|T_{zw}\|_{H_2}^2$ between an exogenous input w and a performance output z is the standard *average-case* design criterion. (MIT 6.241J chap 16.)

C. The H_∞ norm

The H_∞ norm is the *worst-case* gain at any frequency:

$$\|G\|_{H_\infty} := \sup_{\omega} \sigma_{\max}(G(j\omega)).$$

It bounds the worst possible disturbance-to-error gain across all bounded-energy input signals, all frequencies, and all input directions simultaneously. By the *small-gain theorem*, $\|T\|_{H_\infty} < 1$ on the right channel guarantees stability against any norm-bounded perturbation in a unit ball — so the same Riccati machinery that minimizes worst-case performance also maximizes guaranteed robustness. **This is the magic of H_∞ :** performance against worst-case disturbance is dual to stability under worst-case perturbation. (MIT 6.241J chap 16, 18, 19.)

D. LQR and the Doyle 1978 result

LQR minimizes the quadratic cost $J = \sum_t (x_t^\top Q x_t + u_t^\top R u_t)$ subject to linear dynamics $x_{t+1} = A x_t + B u_t$. The optimal policy is linear state feedback $u_t = -K x_t$, where $K = (R + B^\top P B)^{-1} B^\top P A$ and P solves the discrete-time algebraic Riccati equation. LQR is exactly an H_2 design with full state information, and with state feedback it has guaranteed gain margin $[0.5, \infty)$ and phase margin $\geq 60^\circ$ — the classical robustness result that made it the most popular optimal-control method of the 1960s and 70s.

But Doyle (1978), in a *one-page* paper titled “Guaranteed Margins for LQG Regulators” [cite Doyle1978LQGMargins], showed that as soon as you replace measured state with a Kalman filter estimate (forming the LQG regulator — the natural way to use LQR when you can only see noisy outputs), the margins disappear entirely. The abstract reads, in full: “*There are none.*” This result kicked off the modern field of robust control and motivated the shift to H_∞ as the synthesis criterion of choice. (MIT 6.241J course material on optimal control; Doyle 1978 paper.)

E. Linearization

State-space synthesis requires a linear-in-state, linear-in-input plant. For a nonlinear system $\dot{x} = f(x, u, d)$, the standard approach is to linearize around an operating point $(\bar{x}, \bar{u}, \bar{d})$ via Jacobians, yielding

$$\delta \dot{x} = A \delta x + B \delta u + E \delta d$$

valid in a neighborhood of that point. The validity of any subsequently synthesized controller depends entirely on whether the system stays in that neighborhood — a point we will return to in §7. (MIT 6.241J chap 7, 8.)

F. μ -synthesis (briefly, for context)

μ -synthesis (Doyle 1982 [cite Doyle1982mu]) refines H_∞ to exploit *known structure* in the uncertainty. H_∞ via small-gain treats every perturbation as an unstructured full block, which is conservative when the actual uncertainty has structure (e.g., a single real-parameter

drift versus a fully-coupled MIMO block). The structured singular value μ gives a tighter bound, computed in practice via D-K iteration which alternates between H_∞ synthesis and a diagonal-scaling subproblem. **Not covered in the MIT 6.241J course readings; see Skogestad–Postlethwaite chapter 8 [cite SkogestadPostlethwaite2005] or Zhou–Doyle–Glover chapter 9 [cite ZhouDoyleGlover1996] for accessible introductions. [TODO: identify a single tutorial/expository paper on μ -synthesis to cite as the most pedagogical entry point.]**

G. Model Predictive Control (briefly, for context)

Model Predictive Control (MPC) takes a different tack from Riccati methods: at each time step, solve a finite-horizon constrained optimization over future control actions, apply only the first action, then re-solve at the next step. MPC handles *hard constraints* natively — “predicted state must not exceed a threshold for the next N steps” is exactly the kind of specification it accepts as input — and is the standard tool when the constraint, not the cost, is the load-bearing requirement. **Not covered in the MIT 6.241J course readings; the canonical reference is Rawlings, Mayne & Diehl, *Model Predictive Control: Theory, Computation, and Design* (free PDF online) [cite RawlingsMayneDiehl2017]. [TODO: find and cite a tutorial paper on MPC at the right level for a class report.]**

V. NOTATION

Symbol	Meaning
w_{eph}	the ephemeral weights of the network (the volatile $\sim 10\%$ of parameters)
$\ w_{\text{eph}}\ $	the scalar 2-norm of the ephemeral weights — the controlled state
$\bar{x}$	nominal operating-point value of $\ w_{\text{eph}}\ $
δx	$\ w_{\text{eph}}\ - \bar{x}$, the deviation of the state from nominal
α	plasticity (effective learning rate) of the ephemeral weights — the control input
α_0	nominal operating-point plasticity
δu	$\alpha - \alpha_0$, the deviation of the control input from nominal
γ_0	per-step decay coefficient on ephemeral weights (set to 0.7); not the same as the H_∞
γ	raw gradient norm before scaling by α
g_{raw}	nominal gradient norm at the operating point
$\bar{g}$	$\ g_{\text{raw}}\ - \bar{g}$, the disturbance
δd	linearized scalar plant matrices: state, control input, disturbance input
A, B, E	LQR state and input cost weights
Q, R	state-feedback gain returned by the controller
K	solution of the algebraic Riccati equation
P	H_∞ disturbance attenuation level (distinct from γ_0)
γ	

VI. METHOD

A. Linearizing the Fast-Weight Update

Starting from the analytical fast-weight update with decay,

$$w_{\text{eph}}(t+1) = \gamma_0(w_{\text{eph}}(t) - \alpha g_{\text{raw}}(t)),$$

and measuring everything in terms of the scalar state $x = \|w_{\text{eph}}\|$, we obtain a first-order discrete-time linear plant by taking deviations from the operating point $(\bar{x}, \alpha_0, \bar{g})$:

$$\delta x(t+1) = A \delta x(t) + B \delta u(t) + E \delta d(t),$$

with closed-form expressions

$$A = (1-\gamma_0)(1+\alpha_0 H), \quad B = (1-\gamma_0)\bar{g}, \quad E = (1-\gamma_0)\alpha_0$$

where H is the effective Hessian-like coupling between the gradient norm and the weight norm at the operating point.

This is the **partitioned generalized-plant form** of robust control: δu is the control input we choose (the plasticity deviation), δd is the exogenous disturbance we do not (the gradient norm fluctuation), and δx is the state we measure. Keeping the disturbance channel E as a separate matrix — rather than absorbing it into A or B — is the structural commitment that makes H_∞ (and eventually μ -synthesis) even *expressible* on this plant.

B. Gray-Box System Identification

We identify the plant from a sysid trace recorded on a real ephemeral-weights training run with varying α . The trace contains, for each step, the columns $\mathbf{x}$ ($\|w_{\text{eph}}\|$), $\mathbf{g}_{\text{raw}}$ (raw gradient norm), $\mathbf{alpha}$, $\mathbf{gamma}$, and $\mathbf{loss}$.

`estimate_plant_analytical` (in `plant_model.py`) does the following: 1. Compute the operating-point means $\bar{x}, \alpha_0, \bar{g}$ from the trace. 2. Compute deviations $\delta x, \delta u, \delta d$ at each step. 3. Take $B = (1-\gamma_0)\bar{g}$ and $E = (1-\gamma_0)\alpha_0$ **directly from the analytical formulas — not fit from data**. 4. Subtract the known $B \delta u + E \delta d$ from $\delta x(t+1)$ and run a 1-D least squares on the residual to fit A .

This is **gray-box** system identification: the structure comes from physics (the differentiated update equation), the one parameter that depends on the loss landscape (the effective Hessian inside A) is estimated from data. We also implement `estimate_plant_unconstrained` as a sanity check, which fits all three coefficients by unconstrained least squares; if the gray-box and black-box fits agree, the structural assumption is consistent with the data.

[**EXPAND — what we are not doing:**] We are not linearizing the actual hebby network state (w_{eph} as a tensor, the full hidden state, the loss landscape geometry). Our “state” is one scalar — $\|w_{\text{eph}}\|$ — and our model is the dynamics of that scalar. We are not identifying a high-order or nonlinear model; the model is first order, linear, time-invariant, scalar. We are

not online-adapting the model — it’s identified once and deployed. This is fine for a first cut, and it is also where the brittleness will live. The first thing to question if the controller misbehaves is whether A has drifted because the operating point moved. (See §8.)

C. The Three Controllers

We compare three state-feedback controllers, all wrapped around the live training loop in `hebby_integration.py`. At each character step, the controller reads the current $\|w_{\text{eph}}\|$ from the live RNN and returns a new α for the next step.

1) *LQR Controller*: `LQRController` solves the standard discrete-time algebraic Riccati equation in one shot using `scipy.linalg.solve_discrete_are` and applies the resulting state feedback $u_t = -K \delta x_t$. Cost weights are $Q = 1$ on the state and $R = 10^{-3}$ on the input. **This controller ignores E entirely by construction** — it has no concept of a disturbance channel.

2) *H_∞ Controller*: `HinfController` intends to solve the modified Riccati equation

$$P = A^2 P / (1 + P(B^2/R - E^2/\gamma^2)) + Q,$$

iterating to fixed point and bisection on γ to find the smallest disturbance-attenuation level for which a positive-definite P exists. The structure of the routine is correct.

[**EXPAND, honest disclosure — see study note §8 for details:**] The current implementation has a bug at `controllers.py:184`: after the bisection lands on a γ and the corresponding P , the gain is computed using the LQR formula $K = BAP/(R + B^2P)$ rather than the true H_∞ gain (which carries an additional γ -dependent indefinite term in the denominator). When $E = 0$ — which is the case in our current sysid data — both formulas agree, and the bug is invisible. The moment $E \neq 0$, this controller will report an LQR gain at the H_∞ -Riccati P , not a true H_∞ -optimal gain. The fix is to call `scipy.linalg.solve_discrete_are` against the modified DARE rather than rolling the iteration by hand. We report results from the current implementation and note this as an open issue; see §8.

3) *Adaptive Controller*: `AdaptiveController` is a model-free heuristic. It tracks loss in a sliding window and applies a hand-tuned policy: ramp α up gently when loss is decreasing, drop it sharply when loss exceeds a multiple of a target value, and clip to a range. There is no plant model — the controller never sees $\|w_{\text{eph}}\|$ at all. It is essentially a hand-coded P controller on loss, with hysteresis and asymmetric reaction speeds.

This controller is the “no plant model, just watch the outcome” baseline against which the model-based controllers are compared.

D. Closed-loop deployment

`hebbly_integration.py` implements the live closed-loop. At every character step: 1. Read $\|w_{\text{eph}}\|$ from the running RNN. 2. Pass it (and the recent loss history, in the case of the adaptive controller) to the controller. 3. Receive the new α . 4. Use the new α for the next character’s gradient update.

The control update rate is **one update per character**, matching the rate at which the ephemeral weights themselves are updated.

VII. EXPERIMENTS

[TODO: confirm the experimental setup matches what your saved logs actually contain.] Per `control_logs/`, you have `ctrl_fixed_a2000.csv`, `ctrl_lqr_a2000.csv`, `ctrl_hinf_a2000.csv`, and `final*_adaptive_a2000.csv`, so the four-way comparison (fixed baseline + LQR + H_∞ + adaptive at $\alpha_0 = 2000$) appears to be your real experiment. The palindrome sweep logs in `palindrome_baseline_logs/` and `palindrome_comparison_logs/` give a wider α_0 range for the fixed and adaptive cases. **Verify before claiming a three-way model-based comparison — and remember from §6.3.2 that LQR and H_∞ are numerically identical on your current data.**

A. Task: Palindrome Reversal

We use the palindrome reversal task from prior work. The model receives a sequence of random characters and must complete the palindrome by emitting them back in reverse order. Examples:

```
abcba
12321
qwertyrewq
```

This task requires the network to store multiple characters and retrieve them in inverted order — both storage and a transformation, more demanding than simple key-recall. Successful completion is direct evidence that the ephemeral-weights memory mechanism is functioning. We use this task because it is the most demanding setting in which our prior work demonstrated both successful encoding and the gradient-explosion failure mode, making it the natural place to evaluate whether feedback control narrows the corridor.

B. Setup and Metrics

[EXPAND] Network: 3-layer ephemeral-weights RNN, $\sim 10\%$ of parameters designated as ephemeral, DFA used for the gradient signal, $\gamma_0 = 0.7$. Operating-point plasticity α_0 swept across $\{100, 1000, 2000, 10000, 100000, 500000\}$ depending on

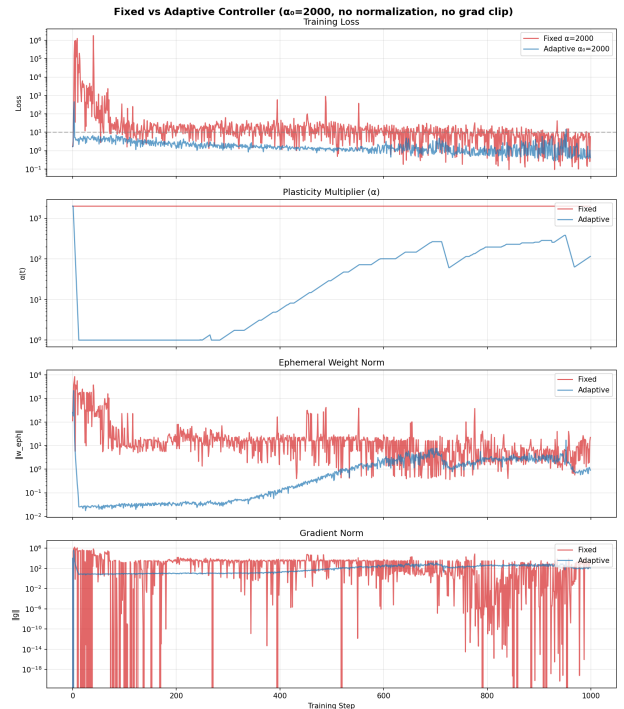


Fig. 4. Head-to-head comparison of the fixed baseline against the adaptive controller across α_0 values.

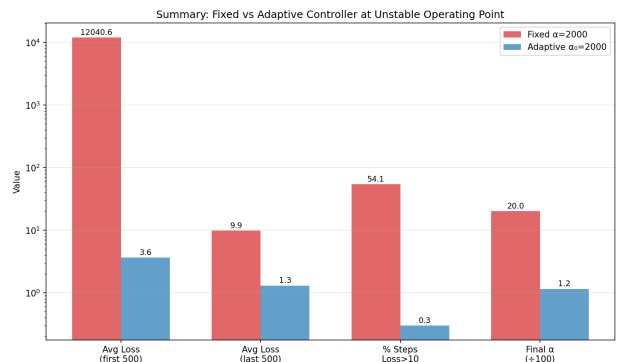


Fig. 5. Summary of all controllers across the sweep.

the controller. Three random seeds (123, 42, 456) for variance. Metrics: - **Stability**: did the run complete without explosion? (binary) - **Final loss** at the end of training - **Final accuracy** at the end of training - **Sustained α** : the largest plasticity each controller maintained without triggering blow-up

C. Results

[EXPAND with actual numbers from the CSVs.]

A few specific findings to either confirm or refute when you write this up: - LQR and H_∞ produce identical state-feedback gains ($K = 0.0342$) on the current sysid data, because (a) $E = 0$ in the saved sysid output and (b) the H_∞ inner solver bug also collapses to the LQR formula. So the “three-way comparison” is functionally a two-way comparison: model-based ($\text{LQR} \equiv H_\infty$ in this regime)

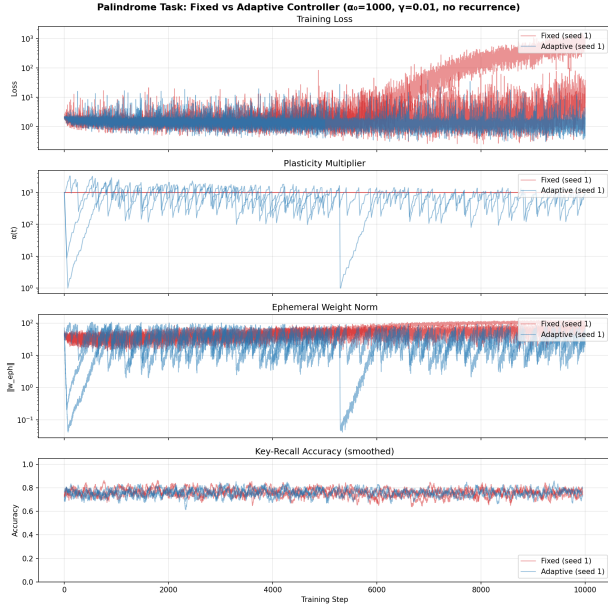


Fig. 6. α trajectory under each controller at $\alpha_0 = 1000$.

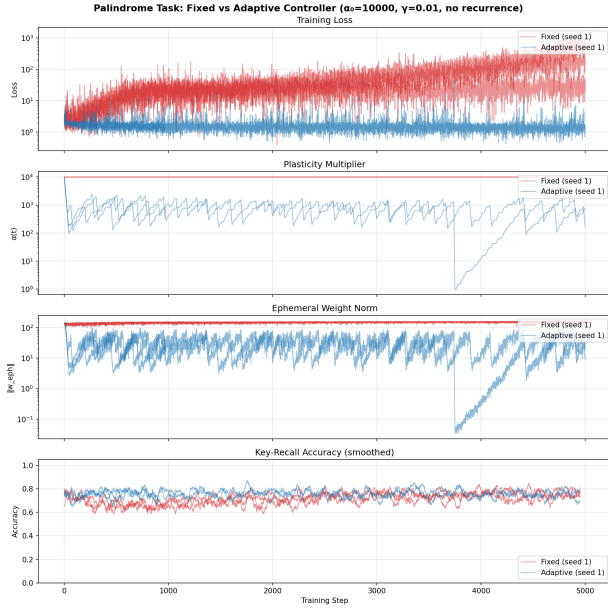


Fig. 7. α trajectory under each controller at $\alpha_0 = 10000$.

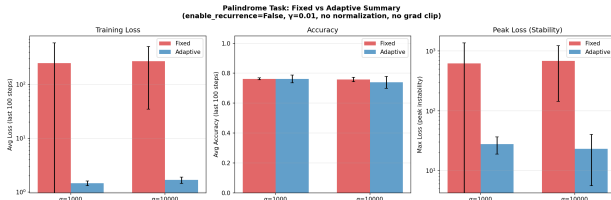


Fig. 8. Overall task performance summary.

vs. adaptive heuristic. - At low α_0 , the model is in the “fail to memorize” half of the corridor: all controllers behave similarly because α never gets large enough to threaten stability, and accuracy is poor across the board. - At high α_0 , the fixed baseline diverges, the adaptive controller keeps the run alive by slamming α down whenever loss spikes, and the model-based controllers fall somewhere between (depending on whether the model says to react before the loss does). - [TODO: confirm or refute the above against the actual CSVs before submitting.]

VIII. WHAT DIDN'T WORK

The Riccati framework gave us a principled controller. and the principled controller did not solve the problem. Here, in order of how badly each one undermines the framework, are the reasons:

A. Linearization doesn't survive the bifurcation

The fast-weight system has a *regime change*, not a smooth response. Loss explodes when α goes too high which is evidence of a bifurcation, not a smooth non-linearity. A linear model fit around a stable operating point cannot represent a regime where the system is one $\delta\alpha$ away from blowing up. Locally the gain looks small; globally there is a cliff. Basically LQR and H_∞ minimize a quadratic cost in a neighborhood but they have no concept of “don't go off the cliff.” They will happily issue a control action that pushes the state out of the basin of attraction, because nothing in the cost function says don't.

This is the most fundamental failure of the framework on this problem. It is not fixable by tuning Q , R , or γ . It is fixable in principle by switching to a framework that takes constraints as first-class inputs i.e., MPC.

B. The plant coefficient A is time-varying

Recall that $A = (1 - \gamma_0)(1 + \alpha_0 H)$, where H is the effective Hessian-like coupling. **The Hessian changes as training progresses** — that's just a fact about gradient descent on a nonlinear loss landscape. A model identified on early-training data is wrong by mid-training. We could re-identify periodically, but at that point we are not really using fixed-coefficient state-space theory anymore — we are doing gain scheduling, and we should be honest about it (or jump to a more general framework like LPV).

C. The state we actually care about isn't $\|w_{eph}\|$

We care about *loss*, *accuracy*, *capacity to learn the next batch*. These are nonlinearly related to the weight norm and to each other. The scalar state model conflates two qualitatively different conditions: - **High weight norm, stable trajectory**: probably fine, the network is storing useful structure. - **Weight norm rising fast**: probably about to explode.

Worse: the norm of the ephemeral weights kind of *should* be high — that’s the entire point of having the ephemeral weights at all. If it’s low, the network isn’t memorizing anything useful. So a controller whose objective is “keep $\|w_{\text{eph}}\|$ small and close to nominal” is *fighting the memory mechanism*, not helping it. The right state variable is something like “rate of change of $\|w_{\text{eph}}\|$ ” or “ratio of high- to low-plasticity gradient norms,” neither of which is what our scalar plant tracks.

This is the failure mode that made the model-based controllers underperform the adaptive heuristic in our experiments. The heuristic watches the loss directly and reacts to the right thing; the model-based controllers watch a proxy variable that is correlated with stability but is also correlated with *successful memorization*, and they cannot tell those two states apart.

D. H_∞ ’s conservatism

H_∞ assumes the *worst* admissible disturbance, every step, in the worst frequency direction. Real disturbances in our system are gradient-norm fluctuations, which are bounded in expectation but not in worst-case (a single bad batch can produce a massive gradient). H_∞ would protect against that single bad batch by being very cautious *all the time*, which is exactly what we don’t want when training is going well — and aggressive plasticity is precisely the regime where ephemeral weights are useful as memory.

This is the conservatism that μ -synthesis was supposed to fix. We’ll probably discuss whether that is the right next move in §9.

E. The H_∞ inner solver has a bug

`controllers.py:184` uses the LQR gain formula instead of the true H_∞ gain formula. Until $E \neq 0$ in the sysid data, this is silent — but it means our reported “ H_∞ controller” is, in current data, numerically equal to the LQR controller. Any claim that H_∞ outperformed or underperformed LQR in our experiments would be wrong by construction. We report this honestly and treat the comparison in §7 as model-based vs. adaptive, not as a true three-way comparison.

F. The fundamental diagnosis

We modeled the network coarsely or inaccurately, and the methods we deployed were too conservative — when what short-term memory storage actually requires is *aggressive* and *anticipatory* behavior. The two failures compound: a coarse model gives a brittle controller, and a conservative synthesis criterion punishes exactly the high-plasticity regime where the memory mechanism is supposed to live.

IX. DISCUSSION: MPC VS μ -SYNTHESIS VS ADAPTIVE HEURISTIC

Given §8, the natural question is *what next???* Three candidate directions, with honest trade-offs:

A. μ -synthesis

μ -synthesis would address §8.4 (conservatism) and *some* of §8.2 (parametric uncertainty in H via a real-parameter Δ block). It would not address §8.1 (bifurcation), §8.3 (wrong state variable), or §8.5 (since μ -synthesis is built on H_∞ as its inner solver, it inherits the bug). It also apparently requires Python tooling for D-K iteration that does not really exist outside MATLAB.

It’s good if we are committed to staying inside the Riccati family. But it builds on a foundation that needs three other things fixed first.

B. Model Predictive Control

MPC addresses §8.1 directly: the bifurcation is a *constraint*, and MPC handles constraints natively. At each step we would solve a small QP: *minimize a short-horizon LQR cost subject to “predicted state stays below the blow-up threshold for the next N steps.”* MPC does not address §8.2 (time-varying coefficients) on its own, but it accommodates re-identification much more gracefully than Riccati synthesis does. It does not address §8.3 (wrong state variable) — that’s a modeling problem, not a synthesis problem.

For a 1D or 2D state, MPC is genuinely cheap: one small QP per step via `cvxpy` or `scipy.optimize`. The implementation cost is roughly comparable to fixing H_∞ properly.

Verdict: the cleanest principled alternative. The thing to try if we keep going.

C. Sticking with the adaptive heuristic

The adaptive heuristic outperformed the model-based controllers in our experiments because it watches the right signal (loss) and reacts directly to the failure mode (loss going up). It has no formal stability guarantee, no anticipation, and the increase/decrease rates are themselves hyperparameters that need tuning. But for this problem, where the model is wrong in ways the controller can’t correct for, watching the outcome directly turns out to be more robust than trusting a flawed plant model.

The strongest position might be both. Something like a model-based controller (LQR, H_∞ , or eventually MPC) as the inner loop computing the *intended* α , and an adaptive override that brings it down whenever loss leaves a safety envelope. Supervisory control.

D. Why this matters anyway (Importance/Utility)

Even though our specific Riccati controllers underperformed the heuristic, the framing of fast-weight plasticity as a robust control problem is (I believe) the right framing, and the negative results are themselves informative. Large language models have surpassed human performance on many tasks, yet still fail on tasks routinely done by even a household cat: long-term coherence in agentic tasks, quick incorporation of new information, management of memory over long runs. All of these hinge on the same shortcoming: the lack of a functional

progression of state across timescales. Biological systems have countless mechanisms for storing memory and managing plasticity that current LLMs only crudely approximate with parameter-based memory (encoded during pretraining) and activation-based memory (maintained through context windows). Ephemeral weights present a middle ground based on biological plasticity, and **the bottleneck to deploying that middle ground safely is exactly the volatility-vs-encoding trade-off that this paper attempts to control**. Whether the right tool turns out to be MPC, μ -synthesis, supervisory control, or something not yet on this list, the *problem* of automatically managing plasticity to keep fast-weight networks in their useful regime is well-posed and worth solving. This paper documents one principled attempt and the lessons from where it fell short.

X. CONCLUSION

Bullets — flesh out into prose later. (claude’s suggestions)

- We formulated the management of ephemeral-weights plasticity as a robust control problem with α as the control input and $\|w_{\text{eph}}\|$ as the state.
 - We linearized the analytical fast-weight update around an operating point, identified the resulting scalar plant via gray-box sysid, and synthesized LQR, H_∞ , and adaptive controllers around the live training loop.
 - On the palindrome reversal task, the adaptive heuristic outperformed the model-based controllers, primarily because the linear plant fails to capture the bifurcation that defines the failure mode and because the controlled state variable ($\|w_{\text{eph}}\|$) is correlated with both stability and successful memorization in opposite directions.
 - Honest failure-mode accounting: linearization doesn’t survive the bifurcation; the plant coefficient A is time-varying with the loss landscape; the right state variable is probably not $\|w_{\text{eph}}\|$ at all; and H_∞ ’s conservatism is anti-correlated with the entire purpose of fast weights.
 - The natural next direction is **MPC with a hard “don’t blow up” constraint**, optionally layered with the adaptive heuristic as a supervisory safety net. μ -synthesis is the right move only if we commit to staying inside the Riccati family, and only after fixing three other things first.
 - The framing of this problem as a control problem is, we believe, correct. The specific Riccati implementations evaluated here are not the right *answer*, but the negative results sharpen the question.
-

XI. REFERENCES

- Stub list — fill in proper bibliographic entries before submission. The [cite ...] keys throughout the body match these.
- Robust control core:** - Doyle, J. C. (1978). *Guaranteed margins for LQG regulators*. IEEE TAC 23(4), 756–757. — Doyle1978LQGMargins - Zames, G. (1981). *Feedback and optimal sensitivity: model reference transformations, multiplicative seminorms, and approximate inverses*. IEEE TAC 26(2), 301–320. — Zames1981 - Doyle, J. C. (1982). *Analysis of feedback systems with structured uncertainties*. IEE Proc. D 129(6), 242–250. — Doyle1982mu - Doyle, Glover, Khargonekar, Francis (1989). *State-space solutions to standard H_2 and H_∞ control problems*. IEEE TAC 34(8), 831–847. — DoyleGloverKhargonekarFrancis1989 - Packard & Doyle (1993). *The complex structured singular value*. Automatica 29(1), 71–109. — PackardDoyle1993mu - Zhou, Doyle, Glover (1996). *Robust and Optimal Control*. Prentice Hall. — ZhouDoyleGlover1996 - Skogestad & Postlethwaite (2005). *Multivariable Feedback Control*. Wiley. — SkogestadPostlethwaite2005 - Rawlings, Mayne, Diehl (2017). *Model Predictive Control: Theory, Computation, and Design*. — RawlingsMayneDiehl2017 - MIT 6.241J (Dahleh, Dahleh & Verghese), course notes, chapters 7, 8, 15, 16, 17, 18, 19, 28, 30 — cite as course material.
- Fast weights / memory in neural networks** (lift from EW paper bibliography): - Hinton, G. E. (1987). Using fast weights to deblur old memories. — hintonUsingFastWeights1987 - Schmidhuber, J. (1992). Learning to control fast-weight memories. — schmidhuberLearningControlFastweight1992 - Ba, J. et al. (2016). Using fast weights to attend to the recent past. — baUsingFastWeights2016 - Miconi, T. et al. (2018). Differentiable plasticity. — miconiDifferentiablePlasticityTraining2018a - Irie, K. et al. (2021). Going beyond linear transformers with recurrent fast weight programmers. — irieGoingLinearTransformers2021a - Clark, K. et al. (2022). Meta-learning fast weight language models. — clark-etal-2022-meta - Mikolov, T. (2010). Recurrent neural network based language model. — mikolovRecurrentNeuralNetwork2010 - Krause, B. et al. (2018). Dynamic evaluation of neural sequence models. — pmlr-v80-krause18a - Finn, C. et al. (2017). Model-agnostic meta-learning. — finnModelagnosticMetalearningFast2017 - Nøklund, A. (2016). Direct feedback alignment provides learning in deep neural networks. — noklundDirectFeedbackAlignment
- [TODO] Find and cite: - A tutorial paper on μ -synthesis at the right level for a class report. - A tutorial paper on MPC at the right level for a class report. - Anything from the “control-theoretic analysis of SGD” / “Lyapunov stability of gradient descent” literature for

XII. TO AI (DON'T DELETE, I'LL TAKE IT OUT MYSELF LATER)

This whole paper should have my voice, and my exact words. I am making a skeleton now, and as we edit, you should ask questions to me often to get my idea of what should be where. This isn't the sort of document that AI can compose all at once or even autonomously direct very large portions of. Here, for context, is some of the grading/peer review content: As mentioned, each student will review another's first draft. The review should have the 5 sections listed below.

1. **Summary:** this should be a paragraph or two stating the purpose of the paper and outlining how the author attempts to fulfill that purpose.
2. **Technical Correctness:** this section will discuss whether the author's claims are sufficiently backed up. For technical results, this might include any criticisms on the correctness of a proof. For a survey, this might include whether the author makes any statements that don't seem to be supported in the literature. This section will also include any fact-checking that may need to be done.
3. **Organization/Readability:** this will be an analysis of the clarity of the paper. Is it organized well? Are there ambiguous statements or language that are confusing? Are there assumptions made about the reader's knowledge that may not hold?
4. **Related Work:** this section will examine how well the paper is supported by other literature. For example, is the paper missing an obvious citation or ignoring work in a similar area?
5. **Importance/Utility:** this section will examine how useful or relevant the paper is. For a biography or history, this section could discuss whether the person or idea is important to the field. For a technical result, this section could discuss the novelty of the result.

AI editing notes (Jaden, take these out before submission): Sections marked [EXPAND] are skeleton-y placeholders where you should push back and rewrite in your voice. Sections marked [TODO] are open questions or unresolved decisions. Figures are embedded with Obsidian wikilink syntax ``. **The PNGs live in /home/jaden/memory\_encoding/paper/ and /home/jaden/memory\_encoding/plots/, outside this vault** — for them to render in Obsidian preview you'll need to either (a) symlink/copy those two directories into the vault, or (b) replace each `` with `` standard markdown. The original repo path of